COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE031	**SECTION:**	CPE21S1
DATE PERFORMED:	10/17/2026	**DATE SUBMITTED:**	10/23/2026
NAME:	Bienvenido B. Pangilinan Jr. Al Twain D. Pasion Carl Andrei D. Torres	**INSTRUCTOR:**	Engr Jimlord Quejado

ACTIVITY NO. 12 Data Dashboard Taxonomies

1. PROCEDURE OUTPUT

TASK 1

CODE

```
import pandas as pd
import numpy as np

np.random.seed(42)

terms = ["Term 1", "Term 2", "Term 3", "Term 4"]
grade_levels = ["Grade 9", "Grade 10", "Grade 11", "Grade 12"]
subjects = ["Math", "Science", "English", "History"]

data = {
    "Student_ID": np.random.randint(1000, 1200, 200),
    "Term": np.random.choice(terms, 200),
    "Grade_Level": np.random.choice(grade_levels, 200),
    "Subject": np.random.choice(subjects, 200),
    "Exam_Score": np.random.randint(50, 100, 200),
    "Attendance_Rate": np.random.randint(70, 100, 200),
    "Study_Hours": np.random.randint(5, 30, 200),
}

df_students = pd.DataFrame(data)
df_students.head()
```

```python
import plotly.graph_objects as go
from plotly.subplots import make_subplots

# Aggregated views
study_hours_by_attendance = df_students.groupby("Study_Hours", as_index=False)["Attendance_Rate"].mean()
exam_score_by_study_hours = df_students.groupby("Study_Hours", as_index=False)["Exam_Score"].mean()
study_hours_trend = df_students.groupby("Term", as_index=False)["Study_Hours"].sum().sort_values("Term")

# Create dashboard layout
fig = make_subplots(
    rows=2, cols=2,
    subplot_titles=(
        "Attendance Rate by Study Hours",
        "Exam Score by Study Hours",
        "Total Study Hours Trend by Term"
    ),
    specs=[
        [{"type": "bar"}, {"type": "bar"}],
        [{"colspan": 2, "type": "scatter"}, None]
    ]
)

# Add charts
fig.add_trace(
    go.Bar(
        x=study_hours_by_attendance["Study_Hours"],
        y=study_hours_by_attendance["Attendance_Rate"],
        name="Attendance Rate",
        marker_color="#1f77b4"
    ),
    row=1, col=1
)

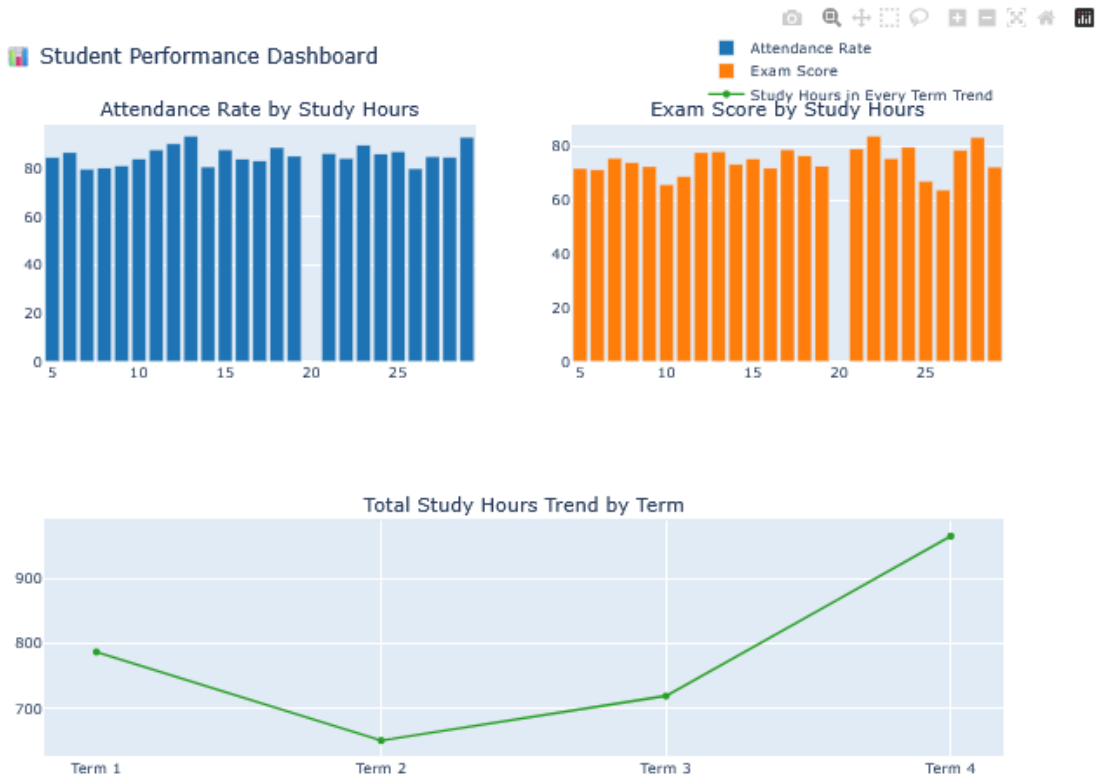

fig.add_trace(
    go.Bar(
        x=exam_score_by_study_hours["Study_Hours"],
        y=exam_score_by_study_hours["Exam_Score"],
        name="Exam Score",
        marker_color="#ff7f0e"
    ),
    row=1, col=2
)


fig.add_trace(
    go.Scatter(
        x=study_hours_trend["Term"],
        y=study_hours_trend["Study_Hours"],
        mode="lines+markers",
        name="Study Hours in Every Term Trend",
        line=dict(color="#2ca02c", width=2)
    ),
    row=2, col=1
)


fig.update_layout(
    height=700,
    width=950,
    title_text="📊 Student Performance Dashboard",
    showlegend=True,
    legend=dict(
        orientation="v",
        yanchor="bottom",
        y=1.02,
        xanchor="right",
        x=1
    )
)

fig.show()
```

OUTPUT



ANALYSIS

PANGILINAN:
Attendance is high for most students, around 80 to 90, while exam scores are lower, around 65 to 85. More study hours do not always mean higher scores. Total study hours went down in Term 2, then went up and reached the highest in Term 4. The dashboard is static, so it only shows the data from when the code was run.

PASION:
For task 1, we created our own dashboard with our own synthesized data set. We demonstrated how to pick appropriate chart types for different kinds of data. All this while also applying the data-to-ink ratio to effectively present the dashboard for the users to analyze, understand, and interpret.

TORRES:
In this task, I've analyzed on how to implement a dashboard using the plotly express. First, we implement a synthetic data by using a (random seed, randint, and randchoice) which generates random data for our synthetic dataset. Next, we implement now the dashboard which we use the plotly express library which we implement first the layout of the dashboard before we create the charts itself. Lastly, we implement the legends which indicates the data that are being presented in the dashboard.

TASK 2

CODE

```python
import time
import numpy as np
import pandas as pd
import plotly.graph_objects as go
from plotly.subplots import make_subplots
from IPython.display import clear_output


def averages(data):
    att = data.groupby("Study_Hours")["Attendance_Rate"].mean()
    exam = data.groupby("Study_Hours")["Exam_Score"].mean()
    return att, exam


def make_button(label, data):
    # one dropdown item that swaps the data in the two bar charts
    a, e = averages(data)
    return dict(label=label, method="restyle",
                args=[{"x": [a.index.tolist(), e.index.tolist()],
                       "y": [a.values.tolist(), e.values.tolist()]}, [0, 1]])


def make_dashboard(df, title):
    # dropdown: All Terms first, then one button per term
    buttons = [make_button("All Terms", df)]
    for t in sorted(df["Term"].unique()):
        buttons.append(make_button(str(t), df[df["Term"] == t]))

    att, exam = averages(df)
    trend = df.groupby("Term")["Study_Hours"].sum()

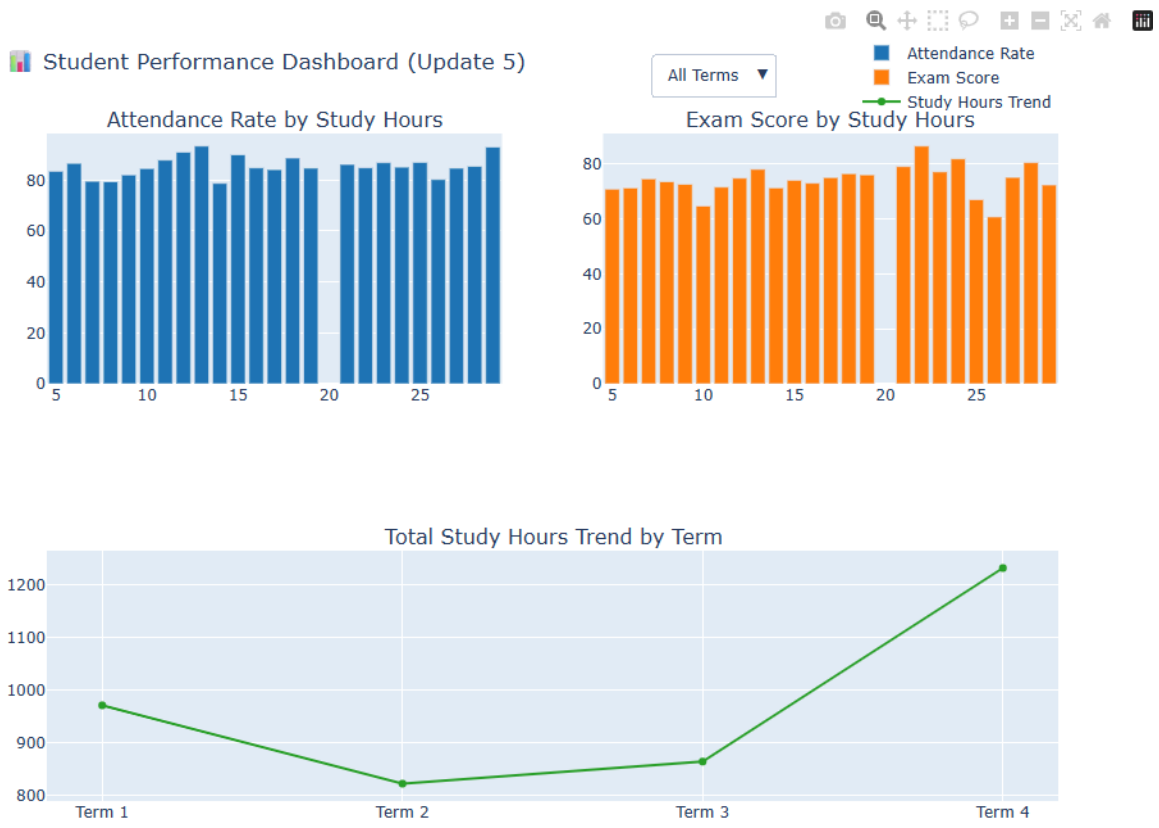
    fig = make_subplots(
        rows=2, cols=2,
        subplot_titles=("Attendance Rate by Study Hours",
                        "Exam Score by Study Hours",
                        "Total Study Hours Trend by Term"),
        specs=[[{"type": "bar"}, {"type": "bar"}],
               [{"colspan": 2, "type": "scatter"}, None]]
    )
    fig.add_trace(go.Bar(x=att.index, y=att.values, name="Attendance Rate",
                         marker_color="#1f77b4"), row=1, col=1)
    fig.add_trace(go.Bar(x=exam.index, y=exam.values, name="Exam Score",
                         marker_color="#ff7f0e"), row=1, col=2)
    fig.add_trace(go.Scatter(x=trend.index, y=trend.values, mode="lines+markers",
                             name="Study Hours Trend",
                             line=dict(color="#2ca02c", width=2)), row=2, col=1)

    fig.update_layout(
        height=700, width=950,
        title_text=title,
        legend=dict(yanchor="bottom", y=1.02, xanchor="right", x=1),
        # dropdown sits next to the legend, on its left
        updatemenus=[dict(buttons=buttons, x=0.72, xanchor="right", y=1.12)]
    )
    return fig

# this one simulate the data in real time (the output may blink couple of time as it updates)
df_live = df_students.copy()

for i in range(1, 6):
    new_rows = df_live.sample(10, replace=True).copy()
    new_rows["Attendance_Rate"] = (new_rows["Attendance_Rate"] + np.random.normal(0, 2, 10)).clip(0, 100)
    new_rows["Exam_Score"] = (new_rows["Exam_Score"] + np.random.normal(0, 3, 10)).clip(0, 100)
    df_live = pd.concat([df_live, new_rows], ignore_index=True)
    clear_output(wait=True)
    make_dashboard(df_live, f"📊 Student Performance Dashboard (Update {i})").show()
    time.sleep(3)
```

OUTPUT



ANALYSIS

PANGALINAN:
The Task 2 dashboard is real-time, but the real-time part is only simulated, like the dashboard updates automatically as new data comes in, which makes it real-time update like every second. The dropdown menu lets the user choose a term, and the charts change to show only that term's data. The user can also hover over the bars to see exact numbers. Because clicking a bar does not show more detail. For organizational function, the dashboard is analytical, because it helps a teacher compare terms and see how study hours relate to attendance and exam scores.

PASION:
For task two, it's slightly different. This is a show of a "Dynamic Dashboard, where it changes over time. In this part of the activity, we simulated the real-time updating by executing a "for loop" that randomizes the dataset and the elements in it within seconds so that we can see the dashboard "update" over time. It is now interactive and can isolate data based on the user. This is more of an operational data with the fact that it uses student performance data to further improve their curriculum.

TORRES:
Task 2 illustrates an up-to-date dashboard by adding 10 random student records every 3 seconds and repeating that for 5 times, thus emphasizing the changes in performance and term trends in real-time streaming data.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

Create a mini dashboard project that tracks student grades or attendance:
1. Include 3 visualizations (e.g., average grades by subject, attendance by week, and performance trend).
2. Add at least one interactive feature (dropdown or slider).
3. Explain how your dashboard helps an instructor or department make data-driven decisions.

CODE

```
# for pie chart
term_attendance = df_students.groupby("Term", as_index=False)["Attendance_Rate"].sum()

# for line chart
term_score = df_students.groupby("Term", as_index=False)["Exam_Score"].mean()

# for bar chart
exam_score = df_students.groupby("Subject", as_index=False)["Exam_Score"].mean()

# Create dashboard layout
fig = make_subplots(rows=2, cols=2,
                    subplot_titles=("Subjects and Average Exam Score", "Attendees per Term", "Average Exam Score per Term", "Temporary"),
                    specs=[[{"type": "bar"}, {"type": "pie"}],
                           [{"colspan": 2}, None]])

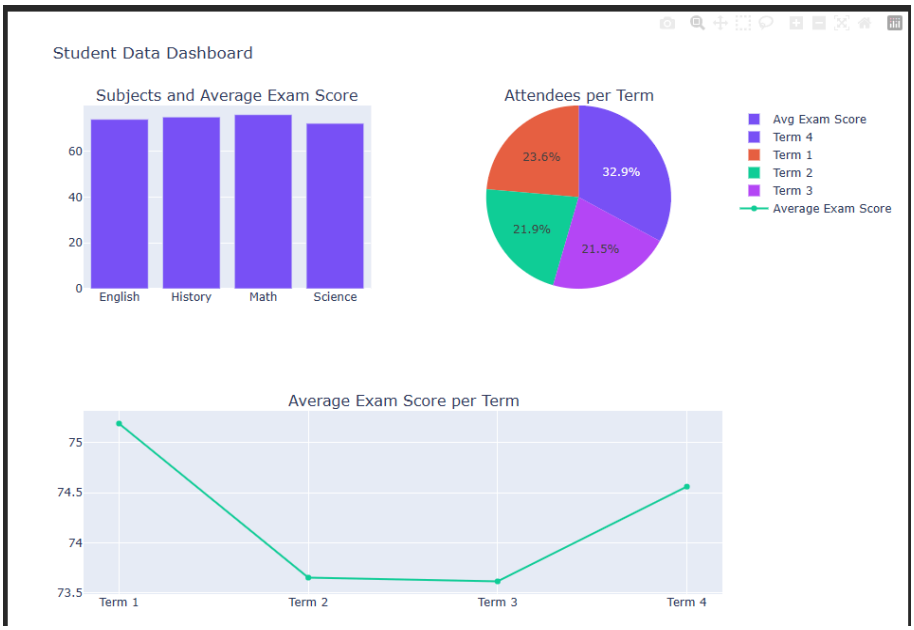
# Add charts
fig.add_trace(go.Bar(x=exam_score["Subject"], y=exam_score["Exam_Score"], name="Avg Exam Score"), row=1, col=1)
fig.add_trace(go.Pie(labels=term_attendance["Term"], values=term_attendance["Attendance_Rate"], name="Term and Attendance"), row=1, col=2)
fig.add_trace(go.Line(x=term_score["Term"], y=term_score["Exam_Score"], mode="lines+markers", name="Average Exam Score"), row=2, col=1)

fig.update_layout(height=700, width=950, title_text="Student Data Dashboard")
fig.show()
```

COMPUTER ENGINEERING

QUEZON CITY

OUTPUT



ANALYSIS

PANGILINAN:
All four subjects have almost the same average exam score. Term 4 has the most attendance, while the other terms are close to each other. Exam scores were highest in Term 1, dropped in Terms 2 and 3, and went up a little in Term 4. The dashboard is static, but hovering shows the exact numbers. It is analytical because it helps an instructor see which terms and subjects need more attention.

PASION:
In the supplementary, our data is static, meaning it doesn't update. With the user interactions, we can specifically see the data we need, for example; we'd want to see the exact average of every exam score each term, we can just hover our cursor to one of the points in the line chart. This dashboard is meant to be analytical, it will show the teaching faculties what they need to improve upon and how to go about their financial decisions based on attendees.

TORRES:
In the supplementary activity, it is the same as task 1 but you need at least one feature. I've analyzed that we want to see the exact average of every exam score on each term, so we can just hover the cursor to one of the points in the line chart to properly see the average. This dashboard is useful for analytical purposes because it will show the faculty members what they need to improve on their teaching approach for their students by analyzing their average of exam scores each term.

CONCLUSION

PANGILINAN:
In this activity, I learned that a good dashboard needs more than nice-looking charts. It should be easy to read, and it should also match how its data is updated, how people use it, and what it is meant for. Now in Task 1, we built a student performance dashboard with Plotly that used different chart types and colors to keep things clear. That dashboard was static, so it only showed the data from the moment the code ran and as for task 2, I made it interactive by adding a dropdown menu to filter by term, and I simulated live data with a loop that adds new records and redraws the charts. One thing I noticed is that every redraw resets the dropdown and makes the output blink a little, which showed me that live dashboards need extra care to feel smooth.

PASION:
In this activity, we went from Non-Interactive Single Graphs to Interactive Dashboards. The topic covered the classifications of dashboards, taxonomy, the design, and how to consider the user's needs when creating a dashboard.

We made static and dynamic dashboards with synthetic data using plotly. Plotly is a much more interactive graph making module compared to matplotlib and seaborn. We can zoom it, hover our mouse to see markers, filter out data, and change how it overall presents itself. For all the tasks, we made synthetic data for student performances. For task 1, it was a presentation of static data, data that is fixed and no updates are made in real time. Task 2 has dynamic data that updates overtime, but here we simulate it by running it through a for loop. Lastly, we chose static for the supplementary that shows the data regarding students, their attendance, and exam scores.

TORRES:
In this hands-on activity, I've learned on how we come up from non-user interactive to a user-interactive charts which we use the plotly express library to make the charts of the dashboard user-interactive by zooming-in, zooming-out, and other functions which makes the dashboard simple to use and understand. In the task 1, I've learned how to create synthetic data by using the random seed to randomly generate data. In the task 2, I've learned how the data are being simulated to streaming data updates in Python. By appending modified copies of the dataset and clearing the notebook output in each iteration, I observed how dashboards dynamically recalculate metrics and re-render visual elements to reflect live changing data. In the supplementary, I've learned how to construct a multi-chart dashboard using Plotly Graph Objects and subplots. By grouping dataset variables by Term and Subject, I was able to structure distinct visualizations—a bar chart for subject averages, a pie chart for attendance distribution across terms, and a trend line for exam performance—into a unified 2 by 2 grid layout with customized column format.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

COMPUTER ENGINEERING

QUEZON CITY

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.